```
!apt-get update -qq
!apt-get install -y -qq libglib2.0-0  # for opencv in some environments
!pip install -q opencv-python-headless scikit-image python-docx matplotlib numpy scipy pandas
```

```
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ul
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 253.0/253.0 kB 8.9 MB/s eta 0:00:00
```

```
import os, glob, math
import numpy as np
import cv2
import matplotlib.pyplot as plt
from skimage import io, img_as_ubyte
from skimage.feature import canny
from skimage.morphology import binary_closing, remove_small_objects, skeletonize, disk
from skimage.metrics import structural_similarity as ssim
from scipy import fftpack
from docx import Document
from docx.shared import Inches
import pandas as pd
%matplotlib inline

results_dir = "/content/q10_results"
os.makedirs(results_dir, exist_ok=True)
print("Results folder:", results_dir)
```

```
Results folder: /content/q10_results
```

```
# CELL 3 — helper functions

def to_gray_float(img):
    # accept path or array
    if isinstance(img, str):
        img = io.imread(img)
    if img.ndim == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    img = img.astype(np.float32)
    if img.max() > 1.0:
        img /= 255.0
    return np.clip(img, 0.0, 1.0)

def fft2(img):
    return fftpack.fftshift(fftpack.fft2(img))

def ifft2(F):
    return np.real(fftpack.ifft2(fftpack.ifftshift(F)))

def ideal_lowpass(shape, cutoff):
    r,c = shape
    R,C = np.ogrid[:r,:c]
    center_r, center_c = r//2, c//2
    mask = ((R-center_r)**2 + (C-center_c)**2) <= cutoff**2
    return mask.astype(float)

def gaussian_lowpass(shape, cutoff):
    r,c = shape
    R,C = np.ogrid[:r,:c]
    center_r, center_c = r//2, c//2
    D2 = (R-center_r)**2 + (C-center_c)**2
    H = np.exp(-D2 / (2*(cutoff**2)))
    return H

def butterworth_lowpass(shape, cutoff, order=2):
    r,c = shape
    R,C = np.ogrid[:r,:c]
    center_r, center_c = r//2, c//2
    D = np.sqrt((R-center_r)**2 + (C-center_c)**2)
    H = 1.0 / (1.0 + (D/cutoff)**(2*order))
    return H

def high_boost_freq(F, H_low, k=1.0):
    H_high = 1.0 - H_low
    G = 1.0 + k * H_high
    return F * G

def image_entropy(img):
```

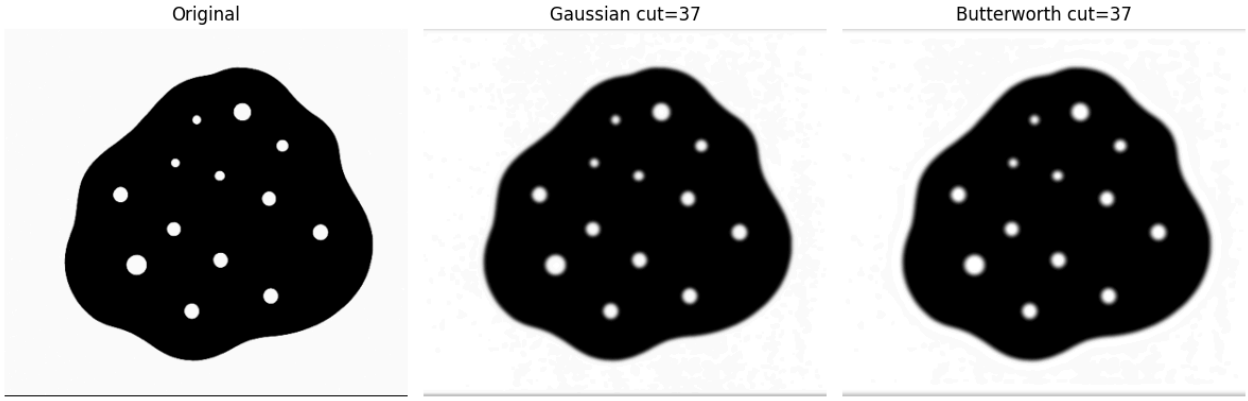
```
    # img float 0..1
    hist, _ = np.histogram((img*255).astype(np.uint8), bins=256, range=(0,255), density=True)
    hist = hist + 1e-12
    return -np.sum(hist * np.log2(hist))

def contrast_std(img):
    return float(np.std(img))

def edge_density(img, sigma=1.0):
    e = canny(img, sigma=sigma)
    return float(e.mean()), e

def save_uint8(path, img):
    io.imsave(path, img_as_ubyte(np.clip(img,0,1)))

def show_img_grid(titles, imgs, cols=2, figsize=(12,6)):
    rows = math.ceil(len(imgs)/cols)
    plt.figure(figsize=figsize)
    for i, im in enumerate(imgs):
        plt.subplot(rows, cols, i+1)
        if im.ndim == 2:
            plt.imshow(im, cmap='gray', vmin=0, vmax=1)
        else:
            plt.imshow(im)
        plt.title(titles[i])
        plt.axis('off')
    plt.tight_layout()
    plt.show()
```

```
# CELL 4 — input image paths (user-provided)
input_paths = [
    "/content/Picture1.png",
    "/content/Picture 2.png",
    "/content/Picture 3.png"
]

# validate
input_paths = [p for p in input_paths if os.path.isfile(p)]
if not input_paths:
    raise FileNotFoundError("No input files found at specified paths. Upload files to /content/ and retry.'

print("Processing files:")
for p in input_paths:
    print(" -", p)
```

```
Processing files:
 - /content/Picture1.png
 - /content/Picture 2.png
 - /content/Picture 3.png
```

```
# CELL 5 — smoothing variants
smoothing_results = {}  # path -> dict
cutoff_fracs = [0.05, 0.10, 0.20]  # try small, medium, large cutoffs

for p in input_paths:
    I = to_gray_float(p)
    r,c = I.shape
    F = fft2(I)
    smoothing_results[p] = {}
    for frac in cutoff_fracs:
        cutoff = max(1, int(min(r,c)*frac))
        H_ideal = ideal_lowpass((r,c), cutoff)
        H_gauss = gaussian_lowpass((r,c), cutoff)
        H_but = butterworth_lowpass((r,c), cutoff, order=2)

        I_ideal = np.clip(ifft2(F * H_ideal), 0, 1)
        I_gauss = np.clip(ifft2(F * H_gauss), 0, 1)
        I_but = np.clip(ifft2(F * H_but), 0, 1)

        key = f"cut{cutoff}"
        smoothing_results[p][key] = {
            'cutoff': cutoff,
            'H_ideal': H_ideal, 'img_ideal': I_ideal,
            'H_gauss': H_gauss, 'img_gauss': I_gauss,
            'H_butter': H_but, 'img_butter': I_but
        }
    # show a preview row (original + gaussian medium + butterworth medium)
    keys = list(smoothing_results[p].keys())
    medium = keys[len(keys)//2]
```

```
    medium = keys[len(keys)//2]
    show_img_grid(
        ["Original", f"Gaussian cut={smoothing_results[p][medium]['cutoff']}", f"Butterworth cut={smoothing_
        [I, smoothing_results[p][medium]['img_gauss'], smoothing_results[p][medium]['img_butter']],
        cols=3, figsize=(12,4)
    )
    # save previews
    save_uint8(os.path.join(results_dir, os.path.basename(p).replace(" ","_") + "_orig.png"), I)
    save_uint8(os.path.join(results_dir, os.path.basename(p).replace(" ","_") + "_gauss_preview.png"), smoot
    save_uint8(os.path.join(results_dir, os.path.basename(p).replace(" ","_") + "_butter_preview.png"), smoo
    print("Saved smoothing previews for", os.path.basename(p))
```



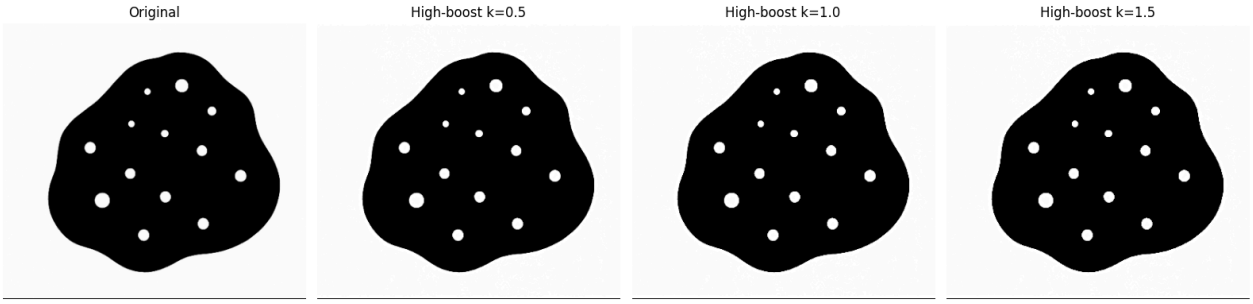

```
Saved smoothing previews for Picture1.png
```


```
Saved smoothing previews for Picture 2.png
```


```
Saved smoothing previews for Picture 3.png
```

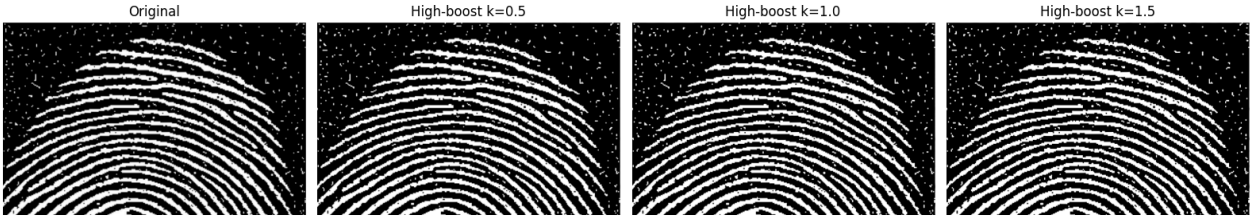
```
# CELL 6 — high-boost sharpening results (k variants)
sharp_results = {}  # path -> dict

k_values = [0.5, 1.0, 1.5]  # mild to stronger boost

for p in input_paths:
    I = to_gray_float(p)
    r,c = I.shape
    F = fft2(I)
    # choose medium cutoff (10% of min dim) to derive H_low
    cutoff = max(1, int(min(r,c)*0.10))
    H_low = gaussian_lowpass((r,c), cutoff)
    sharp_results[p] = {}
    for k in k_values:
        F_sharp = high_boost_freq(F, H_low, k=k)
        I_sharp = np.clip(ifft2(F_sharp), 0, 1)
        sharp_results[p][f"k{k}"] = {'k':k, 'cutoff':cutoff, 'img':I_sharp}
    # show previews (original + k=1.0)
```

```
imgs = [I] + [sharp_results[p]['k{k}']['img'] for k in k_values]
titles = ["Original"] + [f"High-boost k={k}" for k in k_values]
show_img_grid(titles, imgs, cols=4, figsize=(16,4))
# save medium preview
save_uint8(os.path.join(results_dir, os.path.basename(p).replace(" ","_") + "_sharp_k1.png"), sharp_resu
print("Saved sharpening previews for", os.path.basename(p))
```



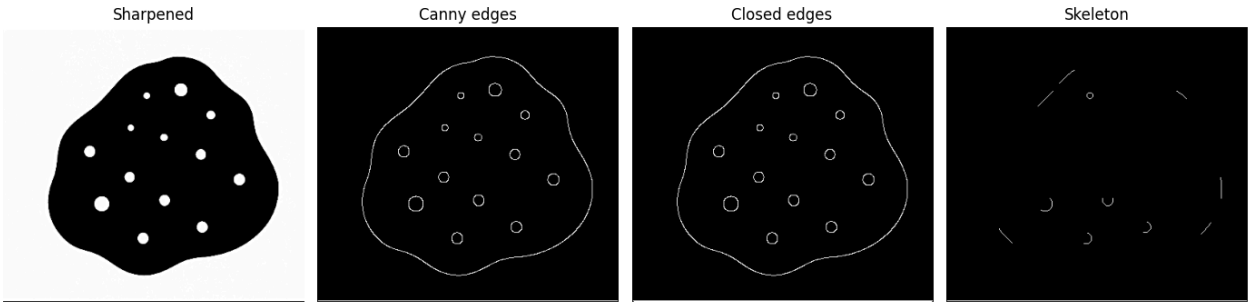

```
Saved sharpening previews for Picture1.png
```


```
Saved sharpening previews for Picture 2.png
```

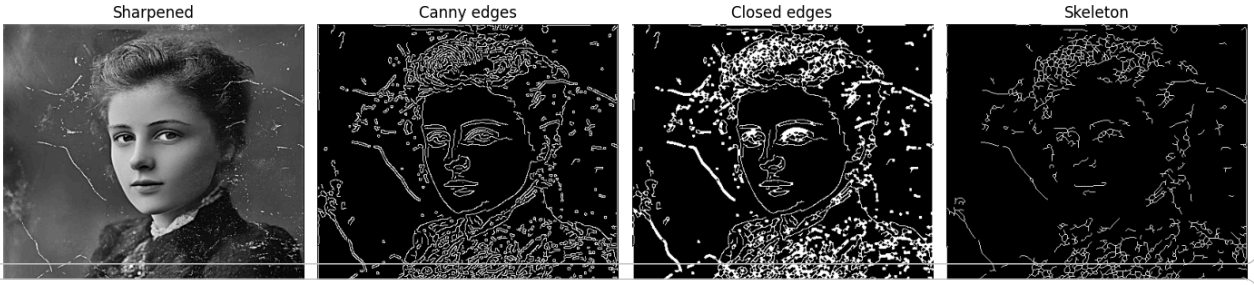



```
Saved sharpening previews for Picture 3.png
```

```
# CELL 7 — boundary extraction and cleaning
boundary_results = {}

for p in input_paths:
    I_sharp = sharp_results[p]['k1.0']['img']
    # Canny edges (tune sigma as needed)
    edges = canny(I_sharp, sigma=1.2)
    edges_closed = binary_closing(edges, disk(1))
    edges_clean = remove_small_objects(edges_closed, min_size=30)
    edges_skel = skeletonize(edges_clean)
    boundary_results[p] = {'raw': edges, 'clean': edges_clean, 'skel': edges_skel}
    # display
    show_img_grid(["Sharpened", "Canny edges", "Closed edges", "Skeleton"],
                  [I_sharp, edges.astype(float), edges_closed.astype(float), edges_skel.astype(float)],
                  cols=4, figsize=(14,4))
    # save
    save_uint8(os.path.join(results_dir, os.path.basename(p).replace(" ","_") + "_boundary_skel.png"), edges
    print("Saved boundary skeleton for", os.path.basename(p))
```




```
/usr/local/lib/python3.12/dist-packages/skimage/_shared/utils.py:328: UserWarning: /content/q10_results/Pic
  return func(*args, **kwargs)
Saved boundary skeleton for Picture1.png
```




```
# CELL 8 — metrics, summary CSV and short docx report
rows = []
doc = Document()
doc.add_heading("Q10 Report — Frequency Filtering, Sharpening & Boundaries", level=1)
doc.add_paragraph("This short report contains previews, metrics and notes for each input image.\n")

for p in input_paths:
    base = os.path.basename(p)
    I_orig = to_gray_float(p)
    # pick gaussian medium smoothing image (10% cutoff)
    smooth_med_key = list(smoothing_results[p].keys())[1]  # second key usually medium
    I_smooth = smoothing_results[p][smooth_med_key]['img_gauss']
    I_sharp = sharp_results[p]['k1.0']['img']
    boundary_skel = boundary_results[p]['skel']

    # metrics
    ent_orig = image_entropy(I_orig)
    ent_smooth = image_entropy(I_smooth)
    ent_sharp = image_entropy(I_sharp)

    std_orig = contrast_std(I_orig)
    std_smooth = contrast_std(I_smooth)
    std_sharp = contrast_std(I_sharp)

    ed_orig, _ = edge_density(I_orig)
    ed_smooth, _ = edge_density(I_smooth)
    ed_sharp, _ = edge_density(I_sharp)

    ssim_sharp = ssim(I_orig, I_sharp, data_range=1.0)

    rows.append({
        'file': base,
        'entropy_orig': ent_orig, 'entropy_smooth': ent_smooth, 'entropy_sharp': ent_sharp,
        'contrast_orig': std_orig, 'contrast_smooth': std_smooth, 'contrast_sharp': std_sharp,
        'edge_orig': ed_orig, 'edge_smooth': ed_smooth, 'edge_sharp': ed_sharp,
        'ssim_sharp_vs_orig': ssim_sharp
    })

    # add to doc: insert preview images (saved earlier) if exist
    doc.add_heading(base, level=2)
    doc.add_paragraph(f"Entropy (orig/smooth/sharp): {ent_orig:.2f} / {ent_smooth:.2f} / {ent_sharp:.2f}")
    doc.add_paragraph(f"Contrast std (orig/smooth/sharp): {std_orig:.4f} / {std_smooth:.4f} / {std_sharp:.4f
    doc.add_paragraph(f"Edge density (orig/smooth/sharp): {ed_orig:.4f} / {ed_smooth:.4f} / {ed_sharp:.4f}")
    doc.add_paragraph(f"SSIM (sharpened vs original): {ssim_sharp:.4f}")
    # attach small images
    # convert in-memory arrays to files then add
    tmp_orig = os.path.join(results_dir, base.replace(" ","_") + "_orig.png")
    tmp_smooth = os.path.join(results_dir, base.replace(" ","_") + "_smooth.png")
    tmp_sharp = os.path.join(results_dir, base.replace(" ","_") + "_sharp.png")
    tmp_bound = os.path.join(results_dir, base.replace(" ","_") + "_boundary.png")
    save_uint8(tmp_orig, I_orig)
    save_uint8(tmp_smooth, I_smooth)
    save_uint8(tmp_sharp, I_sharp)
```

```
    save_uint8(tmp_bound, boundary_results[p]['skel'].astype(float))
    doc.add_paragraph("Previews:")
    doc.add_picture(tmp_orig, width=Inches(2.5))
    doc.add_picture(tmp_smooth, width=Inches(2.5))
    doc.add_picture(tmp_sharp, width=Inches(2.5))
    doc.add_picture(tmp_bound, width=Inches(2.5))

# save CSV and docx
df = pd.DataFrame(rows)
csv_path = os.path.join(results_dir, "Q10_summary.csv")
df.to_csv(csv_path, index=False)
docx_path = os.path.join(results_dir, "Q10_report.docx")
doc.save(docx_path)
print("Saved summary CSV:", csv_path)
print("Saved report DOCX:", docx_path)
```

```
/usr/local/lib/python3.12/dist-packages/skimage/_shared/utils.py:328: UserWarning: /content/q10_results/Pic
  return func(*args, **kwargs)
Saved summary CSV: /content/q10_results/Q10_summary.csv
Saved report DOCX: /content/q10_results/Q10_report.docx
```

```
# CELL 9 — print final table and saved files list
print("Summary table:")
display(pd.read_csv(os.path.join(results_dir, "Q10_summary.csv")))

print("\nFiles saved to:", results_dir)
for f in sorted(os.listdir(results_dir)):
    print(" -", f)
```

```
Summary table:
```

1 to 3 of 3 entries Filter

Copy table

CSV JSON Markdown

Copy

```
index,file,entropy_orig,entropy_smooth,entropy_sharp,contrast_orig,contrast_smooth,contrast_sharp,edge_ori
g,edge_smooth,edge_sharp,ssim_sharp_vs_orig
0,Picture1.png,1.889636469968341,2.622602001487145,2.131978283610621,0.4908857643604278,0.48079178176343,0
.4919072031444856,0.0142258021568366,0.0168419651178271,0.0142058314472107,0.9941768132539716
```

index	file	entropy_orig	entropy_smooth	entropy_sharp	contrast_orig	contrast_smooth	contrast_
0	Picture1.png	1.889636469968341	2.622602001487145	2.131978283610621	0.4908857643604278	0.48079178176343	0.491907203
1	Picture 2.png	7.056304341649971	6.954077040713735	7.216680547211209	0.1479563564062118	0.1362410716951921	0.166407681
2	Picture 3.png	3.4795274178675726	6.51433969107082	2.0331846794911232	0.4027912020683288	0.3237926553041158	0.423930558

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

```
Files saved to: /content/q10_results
 - Picture1.png_boundary.png
 - Picture1.png_boundary_skel.png
 - Picture1.png_butter_preview.png
 - Picture1.png_gauss_preview.png
 - Picture1.png_orig.png
 - Picture1.png_sharp.png
 - Picture1.png_sharp_k1.png
 - Picture1.png_smooth.png
 - Picture_2.png_boundary.png
 - Picture_2.png_boundary_skel.png
 - Picture_2.png_butter_preview.png
 - Picture_2.png_gauss_preview.png
 - Picture_2.png_orig.png
 - Picture_2.png_sharp.png
 - Picture_2.png_sharp_k1.png
 - Picture_2.png_smooth.png
 - Picture_3.png_boundary.png
 - Picture_3.png_boundary_skel.png
 - Picture_3.png_butter_preview.png
 - Picture_3.png_gauss_preview.png
 - Picture_3.png_orig.png
 - Picture_3.png_sharp.png
 - Picture_3.png_sharp_k1.png
 - Picture_3.png_smooth.png
 - Q10_report.docx
 - Q10_summary.csv
```